

ADR-0001 - Synchroniser les salons par SSE

Statut : accepté pour le prototype / à réévaluer avant une montée en charge.

Date : 7 octobre 2026.

Décision : actions HTTP POST et diffusion serveur-vers-navigateur avec Server-Sent Events (SSE).

Contexte

Un salon doit montrer à tous les participants les changements de paramètres, les arrivées, les départs, le lancement et les progrès sans recharger la page. Les modifications partent du navigateur vers le serveur ; les mises à jour du salon repartent du serveur vers tous les navigateurs. Le projet est déjà en Next.js avec des Route Handlers et une base PostgreSQL locale.

Choix retenu

Les actions du joueur utilisent `/api/rooms` et `/api/rooms/[id]` en HTTP POST. Chaque navigateur ouvre `/api/rooms/[id]/events` avec son jeton de membre. Le serveur envoie une vue SSE à l'ouverture puis, environ toutes les 600 ms, relit le salon et publie une nouvelle vue si sa `revision` a changé ; sinon il envoie un commentaire de maintien de connexion. Le client se reconnecte automatiquement si le flux se coupe. Le serveur valide les permissions de l'hôte pour les réglages et le départ.

PostgreSQL conserve chaque salon dans `guepard_rooms.data` (JSONB). Les mutations sont sérialisées par une transaction et un verrou consultatif. Cela évite qu'une invitation privée à usage unique soit consommée simultanément par deux personnes dans ce prototype.

Pourquoi ce choix

SSE correspond au besoin principal de diffusion **du serveur vers les clients** tout en gardant les mutations en POST. Il demande moins de protocole applicatif qu'un WebSocket bidirectionnel pour le fonctionnement actuel et permet d'utiliser les Route Handlers existants. Par rapport à un simple sondage du navigateur, le flux garde une connexion et n'oblige pas chaque composant à gérer son propre intervalle de requêtes.

Solutions écartées pour ce jalon

Option	Raison
WebSocket	Plus flexible pour un échange bidirectionnel permanent, mais infrastructure et gestion de connexion supplémentaires pour ce prototype.
Sondage HTTP côté navigateur	Simple au départ, mais multiplie les requêtes et rend la mise à jour de plusieurs clients moins cohérente.

Option	Raison
Service temps réel externe	Ajoute une dépendance, une configuration et des coûts avant de mesurer la charge réelle.

Conséquences et limites

- Les données du salon restent autoritaires côté serveur ; le client n'écrit pas directement dans PostgreSQL.
- Le flux SSE et la vérification des jetons doivent tourner sur un serveur Node qui accepte des connexions longues. Les limites d'une plateforme d'hébergement doivent être vérifiées avant le déploiement.
- La lecture répétée du salon, son stockage JSONB et le verrou global sont simples mais peuvent limiter la montée en charge. Les tests fonctionnels couvrent une classe d'environ 30 personnes ; ils ne remplacent pas un test de performance multi-salons.
- Avant une production à plus grande échelle : mesurer la latence et les connexions, envisager des lignes par salon/participant et une diffusion par pub/sub. Ne pas présenter le choix actuel comme une architecture déjà éprouvée à grande échelle.

Révision prévue : après les premiers tests de classe et avant la mise en ligne publique.

Sources : `events/route.ts`, `room-client.ts`, `room-store.ts`.